ABSTRACT

Delivering prescribed medicines to patients in a hospital ward is largely a manual task. Staff carry medicines from bed to bed, a box left at the wrong bed is easy to miss, and nobody except the person carrying it knows where a medicine is at a given moment. This project presents **MediDispense**, a low-cost Internet-of-Things system that carries a medicine box to the correct bed automatically and records every step.

A short conveyor belt, driven by a DC gear motor through an L298N motor driver, is controlled by an ESP32 microcontroller. Two infrared sensors detect the box at each bed's pusher line, and two SG90 servo-driven arms, mounted on opposite sides of the belt, push it into the Bed 1 or Bed 2 chute. A push button starts a run, and a buzzer announces the arrival of a job and the delivery of a box. A FastAPI backend stores accounts, consultations, prescriptions and deliveries, authenticates users with JSON Web Tokens and pushes live updates over WebSockets. A single Next.js web application serves patients, doctors, technicians and administrators, and allows a technician to start, stop and test the conveyor directly.

The firmware polls the backend for the next job, dispatches it when the button is pressed, stops the belt at the sensor of the job's own bed, pushes the box and reports its arrival. A timeout and an emergency stop close deliveries that cannot complete. The backend is verified by seven automated end-to-end tests, and delivery to Bed 1 was verified on the hardware with five confirmed clean pushes. The prototype hardware costs about Rs. 2,210.

Keywords: IoT, ESP32, conveyor, medicine delivery, FastAPI, Next.js, hospital automation, embedded systems.

TABLE OF CONTENTS

Chapter 1: Introduction 1

1.1 Background 1

1.2 Problem statement 1

1.3 Objectives 1

1.4 Scope and limitations 2

1.5 Methodology 2

1.6 Organisation of the report 3

Chapter 2: Literature Survey and Existing Approaches 4

2.1 Approaches to in-hospital delivery 4

2.2 Technologies reviewed 4

2.3 Design decisions drawn from the review 5

2.4 Problem gap and positioning 5

Chapter 3: System Requirements and Specification 6

3.1 Overview of the system architecture 6

3.2 Functional requirements 6

3.3 Non-functional requirements 7

3.4 Hardware requirements 7

3.4.1 Development and deployment machine 7

3.4.2 Client devices 7

3.4.3 Conveyor dispensing station 8

3.5 Software requirements 8

3.5.1 Backend server 8

3.5.2 Web application 9

3.5.3 Conveyor firmware 9

3.5.4 Development tools 10

3.6 Communication between components 10

3.7 Justification of technology choices 10
Chapter 4: System Design 12
4.1 System architecture 12
4.2 Mechanical design 12
4.2.1 Layout of pushers, sensors and chutes 12
4.2.2 Arms, risers and chutes 13
4.3 Electronic design 14
4.3.1 Pin assignment 14
4.3.2 Motor driver 14
4.3.3 Sensors 15
4.3.4 Servos, button and buzzer 15
4.3.5 Power and ground rails 15
4.4 Database design 16
4.5 Backend API design 16
4.5.1 API groups and access control 16
4.5.2 Delivery life cycle 17
4.6 Firmware design 17
4.6.1 State machine 17
4.6.2 Safety 18
4.6.3 Finding the backend and the conveyor 18
4.6.4 Local control interface 18
4.7 Web application design 19
4.8 Security design 19
4.9 Complete system flow 20
Chapter 5: Implementation 21
5.1 Hardware implementation 21
5.2 Firmware implementation 21

5.3 Backend implementation 22

5.4 Web application implementation 22

5.5 Integration 22

5.6 Problems encountered and solutions 23

Chapter 6: Testing and Results 24

6.1 Test strategy 24

6.2 Backend tests 24

6.3 Hardware tests 24

6.4 Results against the objectives 25

6.5 Limitations and known issues 25

6.6 Discussion 26

Chapter 7: Conclusion and Future Scope 27

7.1 Conclusion 27

7.2 Future scope 27

References 28

Appendix A: Principal API Endpoints 29

Appendix B: Firmware Controls 30

LIST OF FIGURES

Figure 1: System architecture 12
Figure 2: Top view of the belt: servo 1 pushing the box into the Bed 1 chute (sensor A beam shown in red) 13
Figure 3: Top view of the belt: servo 2 pushing the box into the Bed 2 chute 13
Figure 4: Complete pin diagram of the conveyor station 14
Figure 5: Entities and relationships of the database 16
Figure 6: Life cycle of a delivery 17
Figure 7: Flowchart of the conveyor firmware 18
Figure 8: The complete project flow across the web application, backend and conveyor 20

LIST OF TABLES

Table 1: Approaches to moving medicines inside a hospital 4
Table 2: Design decisions and the reasons for them 5
Table 3: The three components of the system 6
Table 4: Functional requirements 6
Table 5: Non-functional requirements 7
Table 6: Host machine requirements 7
Table 7: Bill of materials of the conveyor dispensing station 8
Table 8: Backend software stack 8
Table 9: Web application software stack 9
Table 10: Conveyor firmware software stack 9
Table 11: Rationale for the principal technology decisions 10
Table 12: ESP32 pin assignment 14
Table 13: Shared power and ground rails 15
Table 14: HTTP routes served by the ESP32 18
Table 15: Web application interfaces by role 19
Table 16: Firmware milestones and their status 21
Table 17: Problems encountered and how they were solved 23
Table 18: Automated backend tests 24
Table 19: Hardware tests and observations 24
Table 20: Objectives and their status 25
Table 21: Principal API endpoints 29
Table 22: Serial console commands used during development 30
Table 23: Firmware constants 30

Chapter 1: Introduction

1.1 Background

A hospital ward runs on timing and accuracy. Every patient has medicines that must reach the bedside at the right moment, and every delivery depends on a member of staff walking the medicine from the point where it is prepared to the bed where it is needed. The task is simple, but it is repeated many times a day and takes nurses away from patient care. It is also difficult to audit: once a box has left the preparation area, the only record of where it is, and whether it arrived, is the memory of the person carrying it.

Embedded systems with built-in networking make it practical to build small connected machines that do such repetitive transport work and report on themselves. The ESP32 microcontroller combines a dual-core processor, WiFi and many general-purpose pins on a board that costs a few hundred rupees. Together with a web backend and a browser-based interface, it allows a delivery machine to receive its own work from a server, carry it out, and report the result, without being tethered to a computer. This project applies that idea to medicine delivery in a ward.

1.2 Problem statement

Manual delivery of medicines inside a ward has three weaknesses:

- **Staff time.** Nurses walk medicines from bed to bed, time that could be spent on patient care.
- **Room for error.** A box left at the wrong bed, or at the wrong time, is easy to miss and hard to trace.
- **No live record.** Doctors and patients cannot see where a prescribed medicine is at any moment.

The problem addressed by this project is to design and build a low-cost system that takes a prescription from the doctor to the correct bed automatically, with a record at every step.

1.3 Objectives

1. Design and build a small conveyor that carries a medicine box to one of two beds.
2. Detect the box at each bed and push it into the correct chute automatically.

3. Develop a backend that stores accounts, prescriptions and deliveries and exposes a documented API.
4. Develop one web application for patients, doctors, technicians and administrators.
5. Connect the conveyor to the backend, so that a prescription triggers a delivery and every step is recorded.
6. Provide safe behaviour: an emergency stop, timeouts, and a way to close deliveries that cannot complete.
7. Verify the system with automated tests and with trials on the hardware.

1.4 Scope and limitations

The prototype is a **model-scale** demonstration. It serves two beds from a 20 cm belt, carries one box at a time and runs on a local WiFi network. It demonstrates the complete chain from prescription to delivery, but it is not a clinical device and has not been validated for use with real medicines. The conveyor's local control interface is intended for a private network and does not require a password. These limits are discussed further in Chapter 6.

1.5 Methodology

The work followed an incremental method. The requirements were first reduced to a design that could be built and tested reliably, then each part was built and tested on its own before it was combined with the others.

- **Design decisions.** The first ideas, a line-following robot, QR-code scanning and an Android application, were replaced by a fixed belt with one pusher per bed, one button and one sensor per bed, and a single web application. Each change removed parts that could fail (Section 2.3).
- **Firmware in milestones.** The firmware was built in six milestones, H1 to H6 (WiFi, belt control, sensors, servos, button and buzzer, and the full delivery loop). Each was tested on the hardware before the next began (Section 5.2).
- **Backend and web in parallel.** The API was written with automated tests, and the web interface was built role by role on top of it.
- **Integration and trials.** The conveyor, backend and web application were then connected and tried together.

1.6 Organisation of the report

- **Chapter 2** reviews existing approaches to in-hospital delivery and the technologies used.
- **Chapter 3** lists the requirements and the specification of the hardware and software.
- **Chapter 4** describes the design: architecture, mechanics, electronics, database, API, firmware and web application.
- **Chapter 5** describes how the system was implemented and the problems met on the way.
- **Chapter 6** presents the tests, the results and the limitations.
- **Chapter 7** gives the conclusion and the scope for future work.

Chapter 2: Literature Survey and Existing Approaches

2.1 Approaches to in-hospital delivery

Medicines and small supplies move inside hospitals in four broad ways. Table 1 compares them in qualitative terms; the comparison reflects general engineering considerations and is not a measurement.

Table 1: Approaches to moving medicines inside a hospital

Approach	How it works	Strengths	Limitations
Manual carrying	Staff carry medicines from the preparation area to each bed.	Flexible, needs no infrastructure.	Uses staff time, relies on memory, leaves no automatic record.
Pneumatic tube systems	Sealed carriers travel through tubes between fixed stations.	Fast and widely used for small items between departments.	Needs installed tubing and stations; does not reach individual beds.
Autonomous mobile robots	Carts navigate corridors and wards on their own.	Flexible routes and large payloads.	Navigation, mapping and obstacle sensing are complex and costly to build and maintain.
Fixed conveyor with bed pushers (this work)	A belt carries the box to a bed's pusher, which pushes it into a chute.	Simple, deterministic, low cost, easy to test.	Fixed route and limited reach; demonstrated here at model scale.

[Add the research papers and commercial systems your team reviewed here, with a one-line summary of each and a citation.]

2.2 Technologies reviewed

- **ESP32 and WiFi.** A low-cost microcontroller with an on-chip WiFi radio, programmable with the Arduino framework. It lets the machine obtain its own work from a server over the network [4].
- **REST and JSON.** The machine and the browser both use the same HTTP API; FastAPI generates the endpoints and validates every request and response [1].
- **WebSocket.** A persistent connection over which the server pushes events to the browser, so dashboards update without refreshing [6].
- **JSON Web Tokens.** A signed, stateless token that identifies a user and role, usable by both the browser and the microcontroller [5].
- **Multicast DNS.** Lets the conveyor be found by a name (conveyor.local) even when the router changes its address [7].

- **Reflective infrared sensing.** An infrared LED and a receiver side by side detect an object by the light it reflects.
- **Hobby servos and PWM.** A pulse-width-modulated signal sets a servo's angle and, at a higher frequency, a DC motor's speed through an H-bridge driver such as the L298N [8].

2.3 Design decisions drawn from the review

Table 2: Design decisions and the reasons for them

Decision	First idea	What was built and why
Delivery mechanism	A line-following robot	A fixed belt with one pusher per bed. There is nothing to navigate, so far fewer parts can fail.
Verification	QR scans of medicine and patient	One button and one IR sensor per bed. Fewer parts and a more repeatable demonstration; the backend still supports scanning as an option.
User interface	An Android application and an admin website	One web application for every role. It runs on any device with a browser and needs no installation.

2.4 Problem gap and positioning

Pneumatic tubes and mobile robots solve delivery at the scale of a whole hospital, at a cost and complexity that is out of reach for a student project. Manual delivery is cheap but leaves no record. This project positions itself between the two: a deliberately simple machine that automates the last step, from a loading point to a bed, and ties it to a prescription record, so that the full chain of doctor, backend, machine and patient can be demonstrated and tested.

Chapter 3: System Requirements and Specification

3.1 Overview of the system architecture

The system is implemented as three independent components that communicate over a local wireless network. Each is written in the language best suited to its platform, and each can be developed, tested and replaced without modifying the others. The dispensing hardware is a network client of the backend rather than a peripheral of a computer, so it needs no physical connection to any laptop during operation. Every person who uses the system does so through the same web application, which presents the pages that belong to their role.

Table 3: The three components of the system

Component	Language	Executes on	Responsibility
Backend API server	Python 3.13	Server / laptop	Data storage, business rules, authentication
Web application	TypeScript	Web browser, any device	Patient, doctor, technician and administrator interfaces, including live conveyor control
Conveyor firmware	C++ (Arduino)	ESP32 microcontroller	Belt control, bed routing, status reporting, local control interface

3.2 Functional requirements

Table 4: Functional requirements

ID	Requirement
FR1	A patient can register themselves with name, age, gender and bed number.
FR2	A patient can see the available doctors and request a consultation.
FR3	A doctor can set their availability, accept requests and see their patients.
FR4	A doctor can write a prescription with one or more medicines for a patient.
FR5	The system creates a delivery for every prescription, for the patient’s bed.
FR6	The conveyor can ask for the oldest waiting delivery.
FR7	An operator presses a button to dispatch the delivery, and the belt carries the box towards the job’s bed.
FR8	The sensor at the job’s bed stops the belt, and the servo at that bed pushes the box into its chute.
FR9	The conveyor reports arrival, and the patient can confirm receipt.
FR10	A delivery that cannot complete (timeout, emergency stop, leftover job) can be closed as failed.
FR11	Jams and failures raise alerts that a technician can acknowledge and resolve.
FR12	A technician can start, stop, push and beep the conveyor directly from the web application.

ID	Requirement
FR13	An administrator can manage staff accounts and view records and reports.
FR14	Access to every function is limited by the user's role.

3.3 Non-functional requirements

Table 5: Non-functional requirements

Quality	Requirement
Safety	The belt must never run unattended: an emergency stop, a run timeout and a restart on WiFi loss stop it.
Reliability	Sensors and the button are debounced; calls to the backend have timeouts and a retry on arrival reporting.
Security	Passwords are hashed, logins use signed tokens, and every endpoint checks the caller's role.
Usability	Everything is reachable from a browser, with no application to install.
Responsiveness	The conveyor asks for a job every 4 seconds; the control panel refreshes every 1.5 seconds.
Maintainability	Independent components, a documented API and automated tests.
Cost	A parts cost low enough for a student prototype.

3.4 Hardware requirements

3.4.1 Development and deployment machine

The backend server and the web application are hosted on a standard personal computer.

Table 6: Host machine requirements

Item	Specification
Processor	Intel Core i3 (or equivalent) and above
Memory (RAM)	4 GB minimum, 8 GB recommended
Storage	10 GB of free disk space
Network	Wi-Fi adapter, 2.4 GHz capable (required by the ESP32)
Operating system	Windows 10/11, Linux or macOS

3.4.2 Client devices

The web application needs only a device with a current web browser (Google Chrome, Microsoft Edge or Mozilla Firefox) and a network connection to the host machine. Nothing is installed on the client. To control the conveyor directly from the Technician page, the browser must be on the same WiFi network as the conveyor, because that page talks to the station over the local network rather than through the backend.

3.4.3 Conveyor dispensing station

Table 7: Bill of materials of the conveyor dispensing station

Sl.	Component	Qty	Cost (Rs.)	Function in the system
1	ESP32 DevKit V1 (ESP-WROOM-32, 30-pin)	1	399	Microcontroller with integrated Wi-Fi; runs the station firmware
2	L298N dual H-bridge motor driver	1	131	Supplies switched current to the belt motor
3	Conveyor belt kit (frame, rubber belt, pulleys, geared motor)	1	499	Mechanical structure and drive motor (20 cm belt)
4	BO geared DC motor (55-110 RPM)	1	41	Spare drive motor; the kit's own motor was used
5	SG90 micro servo motor	2	186	Pusher arm at each bed station
6	IR obstacle-detection sensor	2	276	Detects the box arriving at each pusher line
7	Active buzzer module	1	42	Audible indication of job arrival and delivery
8	Tactile push button	1	47	Operator confirmation that the box is loaded (pack of 10)
9	LM2596 buck converter	1	45	12 V to 5 V for standalone operation; not used while the ESP32 is powered over USB
10	12 V 2 A DC adapter	1	389	Power source for the motor driver and belt motor
11	Jumper wires (M-M, M-F, F-F)	1 set	130	Interconnection between modules
12	USB-A to Micro-USB cable	1	25	Firmware upload and power during development
	Total		**2,210**	

Two IR sensors are specified rather than one. With a single sensor the firmware could not tell the arrival position of one bed from another and would have to rely on belt timing, which drifts as the belt stretches or the box slips. One sensor per pusher line removes that dependency. The servo arms, risers and bed chutes are made from locally sourced plastic sheet, cardboard and adhesive, which are inexpensive and are not included in the total.

3.5 Software requirements

3.5.1 Backend server

Table 8: Backend software stack

Technology	Version	Purpose
Python	3.13	Implementation language
FastAPI	0.115.6	Web framework; defines all REST endpoints and WebSocket channels
Uvicorn	0.32.1	ASGI server that executes the FastAPI application

Technology	Version	Purpose
SQLAlchemy	2.0.36	Object-relational mapper; tables are declared as Python classes
SQLite / MySQL	PyMySQL 1.1.1	Database engine: SQLite in development, MySQL for deployment
Alembic	1.14.0	Database schema migration
Pydantic	v2	Validation of every request and response
python-jose	3.3.0	Generation and verification of JWT tokens
passlib with bcrypt	1.7.4	One-way password hashing
qrcode with Pillow	8.0	Patient QR images (retained; not required by the minimal flow)
firebase-admin	6.6.0	Push notifications (retained from an earlier design; unused by the web application)
pytest	8.3.4	Automated regression tests

3.5.2 Web application

Table 9: Web application software stack

Technology	Version	Purpose
TypeScript	5.x	Implementation language with static type checking
Next.js	16.3.0	React framework providing routing and build tooling
React	19.2.8	User interface library
Tailwind CSS	4.x	Styling
Axios	1.19	HTTP client for the backend API
Recharts	3.10	Charts on the Reports page
jwt-decode	4.0	Reads the token held in browser storage

3.5.3 Conveyor firmware

Table 10: Conveyor firmware software stack

Technology	Version	Purpose
C++ (Arduino framework)	-	Implementation language
ESP32 Arduino core	3.3.11	Board support package for the ESP32
arduino-cli	1.5.1	Compilation and upload of the firmware
WiFi, HTTPClient	core library	Network connection and REST calls to the backend
WebServer, ESPmDNS	core libraries	Local control interface and the name conveyor.local
ESP32Servo	3.2.1	Control of the two pusher servos
ArduinoJson	7.4.3	Parsing and building JSON

3.5.4 Development tools

Visual Studio Code for the backend and web application, the Arduino framework with arduino-cli for the firmware, Git for version control, browser developer tools for inspecting the web application, and Postman or cURL for manual API tests.

3.6 Communication between components

The three components exchange data using three mechanisms, all over the local wireless network:

- **REST over HTTP with JSON** is used by the web application and the conveyor firmware to read and change data held by the backend.
- **WebSocket** is a persistent connection from the web application to the backend, over which delivery and alert events are pushed so that dashboards update without being refreshed.
- **A local control interface**: the conveyor firmware also runs a small web server, reachable by the name conveyor.local. The technician's browser uses it to start and stop the belt, trigger a pusher or sound the buzzer directly.

Every delivery is created, dispatched and completed through the backend, so the stored record is complete and consistent regardless of which client initiated the action. The one exception is manual control: the technician's browser may command the conveyor directly for commissioning and testing, and such commands neither create nor alter delivery records.

3.7 Justification of technology choices

Table 11: Rationale for the principal technology decisions

Choice	Justification
ESP32 in place of Arduino Uno	The ESP32 has an integrated Wi-Fi radio, so the station obtains its own work from the backend and needs no USB connection to a computer during operation.
FastAPI in place of Flask	FastAPI validates requests and responses automatically through Pydantic, generates interactive API documentation, and supports WebSockets and asynchronous handling natively.
Web application in place of a native mobile application	A browser interface runs on any device without installation, serves every role from one code base and is reachable from any workstation on the network.
SQLite with MySQL compatibility	SQLite needs no server installation. Because all access goes through SQLAlchemy, moving to MySQL needs only a change of connection string.

Choice	Justification
JSON Web Tokens	A stateless token serves the browser and the microcontroller alike without server-side session storage.
Fixed belt with two pushers in place of a mobile robot	It removes navigation, line following and obstacle sensing, the least reliable parts of a robot, while still demonstrating automatic routing to the correct bed.

Chapter 4: System Design

4.1 System architecture

Figure 1 shows the three components. The backend is the single source of truth. The web application is how every person uses the system and receives live updates over a WebSocket. The conveyor firmware logs in to the backend as a technician account, asks for the next job and reports back. The amber path is the direct local control used for testing.

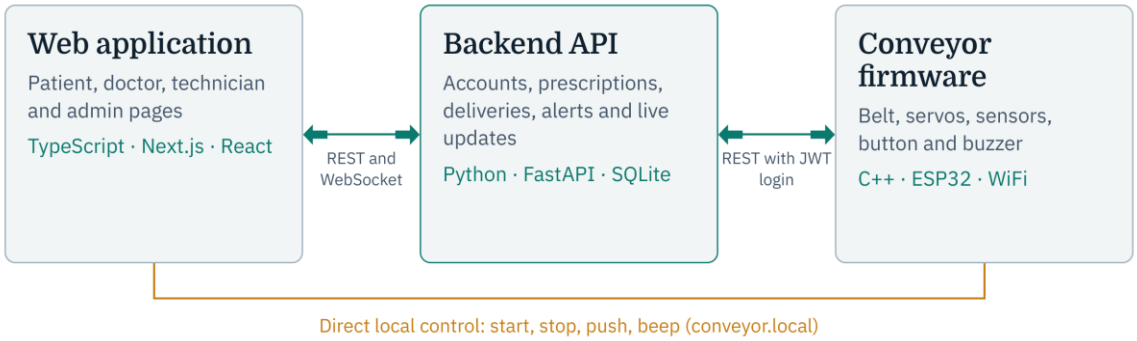


Figure 1: System architecture

4.2 Mechanical design

The mechanical base is a commercial wooden conveyor kit with a 20 cm by 6.5 cm rubber belt and its own small DC gear motor. The kit is assembled as a plain belt first, and the pushers, sensors and chutes are added around it.

4.2.1 Layout of pushers, sensors and chutes

Each bed has one pusher line. A servo with a plastic arm stands outside one edge of the belt and swings the arm across the belt to push the box off the opposite edge into that bed's chute. The arm needs a reach of about 7.5 cm, which is longer than the 6.5 cm spacing between the two pusher lines. If both servos were on the same side, the parked arm of the first would collide with the second. **The two servos are therefore placed on opposite sides of the belt**: servo 1 on the near side pushes into the Bed 1 chute on the far side, and servo 2 on the far side pushes into the Bed 2 chute on the near side.

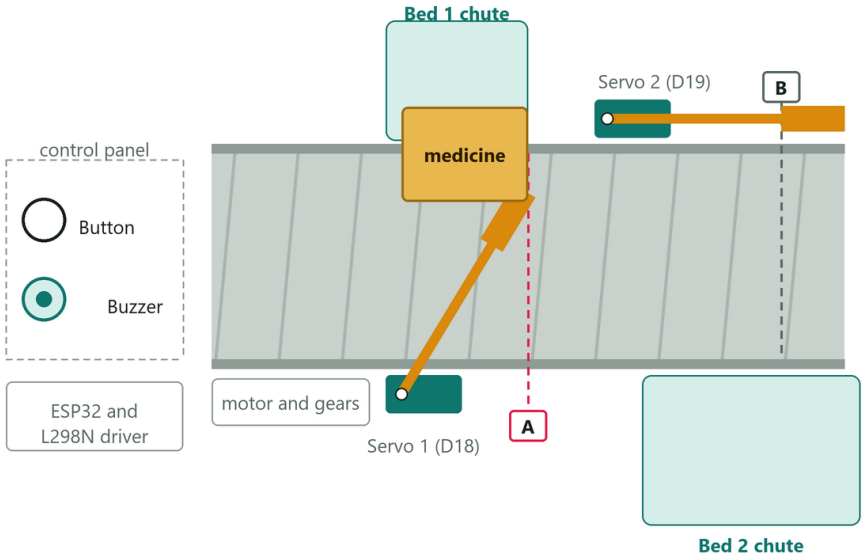


Figure 2: Top view of the belt: servo 1 pushing the box into the Bed 1 chute (sensor A beam shown in red)

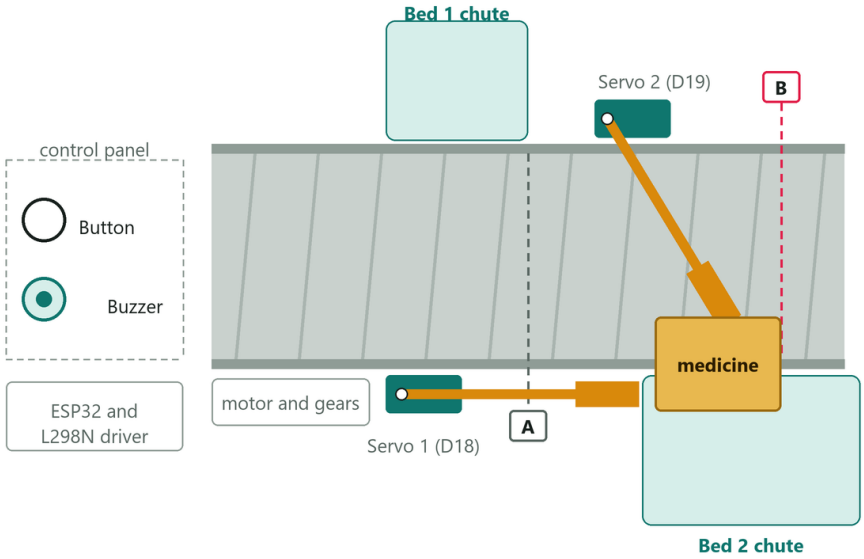


Figure 3: Top view of the belt: servo 2 pushing the box into the Bed 2 chute

Each infrared sensor is placed just upstream of its servo's sweep zone, so that the box is within reach of the arm when the sensor sees it. The control panel (button and buzzer) and the electronics sit at the loading end, away from the arms' sweep. The test box is about 4 cm by 3 cm by 1.5 cm, which clears the belt width with margin.

4.2.2 Arms, risers and chutes

Each arm is made of three layers of thin plastic sheet glued together for stiffness, with an L-shaped paddle fixed under its tip so that the paddle's flat face pushes the box. Each servo stands on a hollow riser folded from cardboard, so the paddle hangs a few

millimetres above the rail yet still meets the box. The chutes are cardboard trays on the belt's sides.

4.3 Electronic design

4.3.1 Pin assignment

Table 12 lists every signal and the ESP32 pin it uses, and Figure 4 shows the complete connections.

Table 12: ESP32 pin assignment

Device	Wire	ESP32 pin
Sensor A (Bed 1)	S, orange	D4
Sensor B (Bed 2)	S, orange	D32
Push button	one leg (the other to ground)	D15
Buzzer	SIG, black	D23
Servo 1 (Bed 1)	signal, yellow	D18
Servo 2 (Bed 2)	signal, yellow	D19
L298N	ENA (speed, PWM)	D25
L298N	IN1 (direction)	D26
L298N	IN2 (direction)	D27

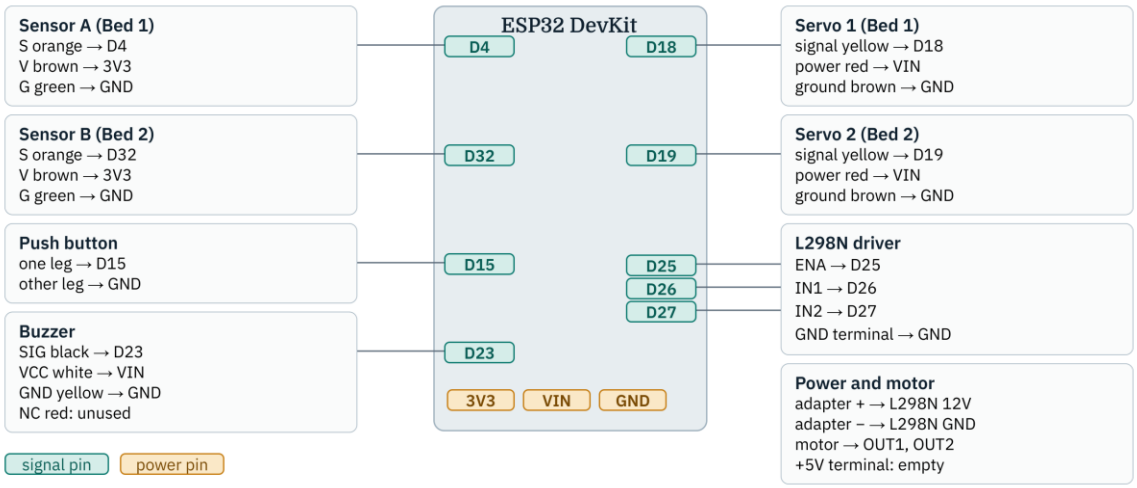


Figure 4: Complete pin diagram of the conveyor station

4.3.2 Motor driver

The ESP32's pins cannot power a motor, so the belt motor is switched by an L298N dual H-bridge driver. The ENA input takes a PWM signal that sets the belt speed, and IN1 and IN2 set the direction. The driver's ENA jumper must be removed, otherwise the enable input is tied high and the speed cannot be controlled. The belt runs at a PWM frequency of 1 kHz and a duty setting of 90 out of 255; frequencies of 15 to 20 kHz only made the

motor buzz and did not turn it. The motor draws its current from a separate 12 V adapter connected to the driver's 12V and GND terminals, and the driver's ground is joined to the ESP32's ground.

4.3.3 Sensors

Each sensor is a reflective infrared module: an infrared LED sends out light and a receiver beside it watches for the reflection. A box in front reflects the light back and the output pin goes LOW. The firmware samples each sensor every 10 ms and accepts a change only after five consecutive readings agree, so a flicker is never counted as a box. The sensors run from 3.3 V so that their signal is safe for the ESP32 pin. The modules' small connector made poor contact with jumper wires, so wires were soldered directly to the module's pins.

4.3.4 Servos, button and buzzer

The SG90 servos take a PWM pulse that sets the arm's angle through the ESP32Servo library; the arm rests at 0 degrees and swings to 90 degrees to push (the angles are firmware settings and can be calibrated). The button is wired between pin D15 and ground and uses the ESP32's internal pull-up, so the pin reads HIGH until the button is pressed; it is debounced in software. The active buzzer sounds while pin D23 is HIGH: a short beep marks a job arriving or a button press, a longer beep marks a delivery, and three quick beeps mark an error.

4.3.5 Power and ground rails

The ESP32 has two ground pins, one 5 V (VIN) pin and one 3.3 V pin, but more parts need power and ground than there are pins. The wires are therefore joined into four shared rails, each taken to one ESP32 pin. The grounds are split so that the servos and the motor driver, which draw current in pulses, do not share a ground path with the sensitive sensors.

Table 13: Shared power and ground rails

Rail	ESP32 pin	Wires joined on the rail
Ground one	one GND pin	Sensor A and B ground (green), button's other leg, buzzer GND (yellow)
3V3	3V3	Sensor A and B power (brown)
Ground two	the other GND pin	Servo 1 and 2 ground (brown), L298N GND
VIN (5 V)	VIN	Servo 1 and 2 power (red), buzzer VCC (white)

The 12 V adapter powers only the motor driver and the belt motor; only its ground is shared with the ESP32. During development the ESP32 is powered over USB.

4.4 Database design

The backend stores its data in a relational database, defined with SQLAlchemy. Figure 5 shows the main entities and their relationships. A user has a role (administrator, pharmacist, nurse, doctor, technician or patient). A patient record holds the bed number and may be linked to the patient's login account. A prescription belongs to one patient and one doctor, may come from a consultation, and has one or more medicines. Each prescription has exactly one delivery, whose status tracks the run. Alerts refer optionally to a delivery, and a single conveyor status record holds the belt's state.

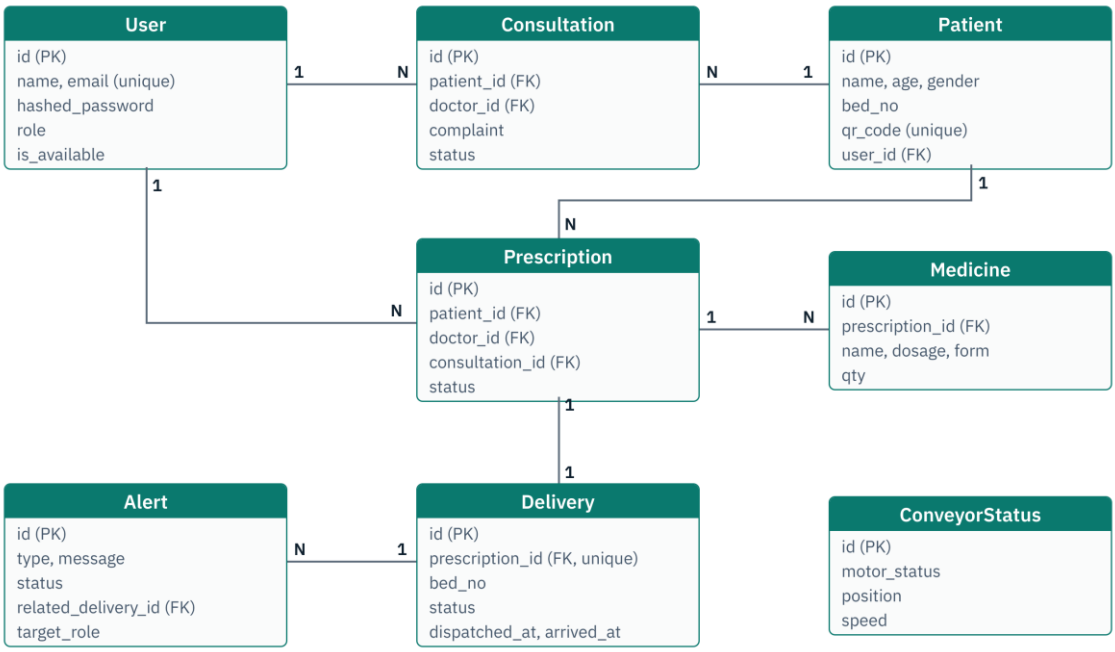


Figure 5: Entities and relationships of the database

4.5 Backend API design

4.5.1 API groups and access control

The API is grouped by resource: authentication, users, patients, consultations, prescriptions, deliveries, conveyor, alerts and reports, plus a WebSocket endpoint with channels for the conveyor, alerts and prescriptions. Every endpoint declares which roles may call it, and any other caller is refused. The principal endpoints are listed in Appendix A.

4.5.2 Delivery life cycle

A delivery moves through the states in Figure 6. It is created as **preparing** when the doctor writes the prescription. The conveyor's button press **dispatches** it (in transit), the conveyor's report after the push marks it **arrived**, and the patient's confirmation makes it **confirmed**. A delivery that is still open can be closed as **failed**. Each step is checked: for example a delivery cannot be dispatched twice, cannot be confirmed before it has arrived, and a finished delivery cannot be failed.

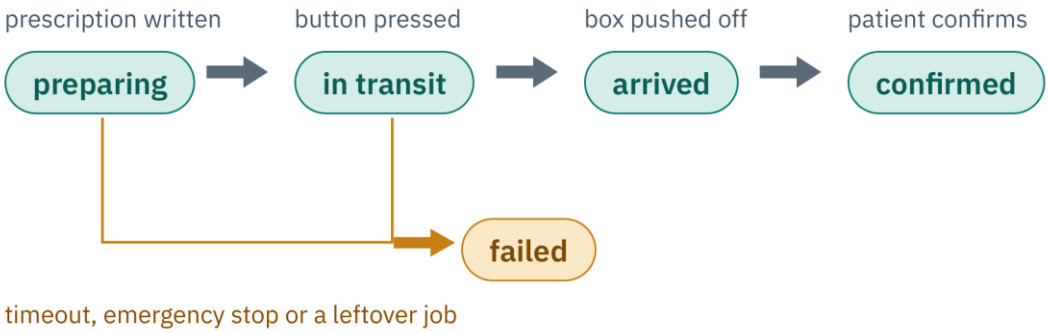


Figure 6: Life cycle of a delivery

The conveyor obtains its work from the **next-job** endpoint, which returns the oldest delivery still preparing (first in, first out), or nothing when there is none, so the operator is never unsure which box to load next.

4.6 Firmware design

4.6.1 State machine

The firmware is a small state machine with three states. In **Idle** it asks the backend for the next job every four seconds. When a job arrives it beeps and enters **Loading**, where the operator loads the box and presses the button; the press dispatches the delivery in the backend and starts the belt, entering **Running**. In Running the belt moves until the sensor at the job's own bed sees the box; the other bed's sensor is ignored, so a Bed 2 box rides past sensor A. The firmware then stops the belt, pushes with that bed's servo, beeps, marks the delivery arrived and returns to Idle.

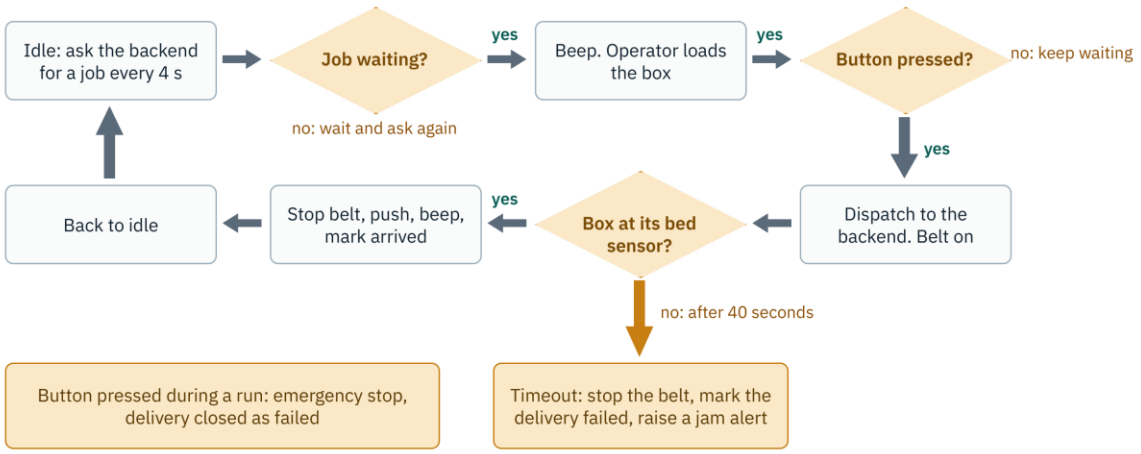


Figure 7: Flowchart of the conveyor firmware

4.6.2 Safety

- Pressing the button during a run is an **emergency stop**: the belt stops, a jam alert is raised and the delivery is closed as failed.
- If no box reaches its sensor within **40 seconds**, the belt stops, a jam alert is raised and the delivery is closed as failed.
- If the WiFi connection is lost, the belt is stopped and the board restarts, so the belt never runs unattended.
- Every call to the backend has a connect timeout of 3 seconds and a read timeout of 5 seconds, so a network problem cannot freeze the station.

4.6.3 Finding the backend and the conveyor

The router assigns new addresses from time to time, so neither side relies on a fixed address. The firmware looks up the backend's computer name with multicast DNS and falls back to the address stored in its configuration file; the web application reaches the conveyor as conveyor.local and offers an editable address if that name fails.

4.6.4 Local control interface

Table 14: HTTP routes served by the ESP32

Route	Method	Action
/status	GET	Belt state, speed, both sensors, servo angles, WiFi signal, address, mode and current job
/start	POST	Run the belt
/stop	POST	Stop the belt
/push?bed=1 or 2	POST	Run that bed's servo push
/beep	POST	Sound the buzzer

4.7 Web application design

The web application is a Next.js application written in TypeScript. After login, each user is taken to the pages for their own role, and pages that belong to other roles cannot be opened.

Table 15: Web application interfaces by role

Role	Pages	Main functions
Patient	Sign-up, home, find a doctor, my consultations	Registers with a bed number, requests a consultation, follows the medicine on its way and confirms receipt
Doctor	Consultations	Sets availability, accepts requests, writes prescriptions, and sees patients and the live status of each delivery
Technician	Conveyor	Monitors the belt and alerts, controls the conveyor directly, and closes stuck deliveries
Administrator	Dashboard, patients, staff, prescriptions, consultations, alerts, reports	Manages staff accounts, oversees every record and delivery, and can also control the conveyor

The **hardware control panel** is shown on the Technician page and on the administrator's dashboard. It refreshes the conveyor's status every 1.5 seconds and shows whether the conveyor is online, the belt state, both sensors, the WiFi signal and the current mode and job. Buttons start and stop the belt, push at Bed 1 or Bed 2, and sound the buzzer. Nurse and pharmacist accounts exist in the data model but have no dedicated page; the operator's confirmation is the physical button on the station.

4.8 Security design

- Passwords are hashed with bcrypt and never stored in plain text.
- Logins return a signed JSON Web Token that expires after 12 hours; the conveyor logs in again when its token expires.
- Every endpoint checks the caller's role; a patient can see only their own records and deliveries.
- The conveyor's configuration file, which holds the WiFi password and the machine's login, is kept out of version control; an example file with placeholders is provided instead.
- The conveyor's local control interface uses plain HTTP without a password and is intended for a private network only (Section 6.5).

4.9 Complete system flow

Figure 8 follows one prescription through the three components, from the patient's sign-up to the confirmation of receipt. Every step is recorded in the backend and shown on the dashboards.

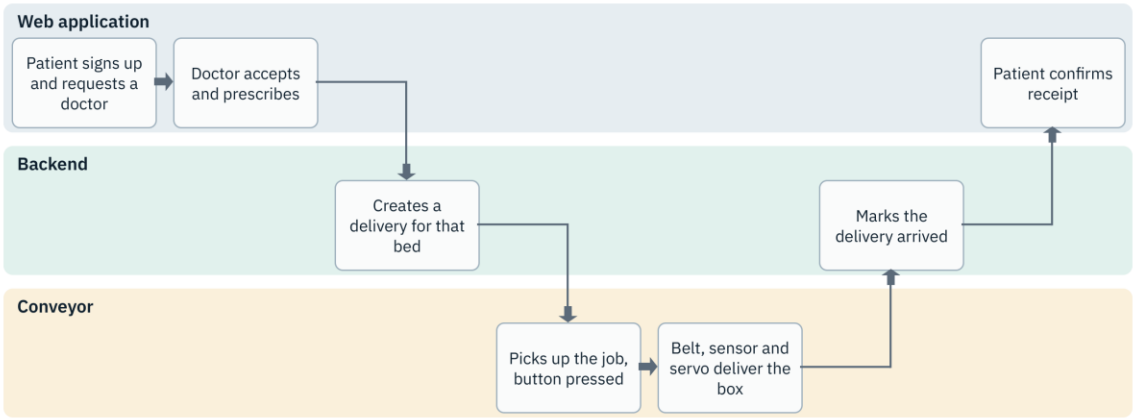


Figure 8: The complete project flow across the web application, backend and conveyor

Chapter 5: Implementation

5.1 Hardware implementation

- **Belt assembly.** The kit was assembled as a plain belt, and the belt ran smoothly and quietly on its own motor before anything was added.
- **Motor control.** The kit's battery holder was removed and the motor's wires were taken to the L298N output terminals. The driver's ENA jumper was removed and its enable pin wired to D25. The motor wires worked loose twice during testing; the ends were cut to bare copper, clamped fully under the terminal screws and strain-relieved.
- **Sensors.** The infrared modules' connector did not make reliable contact with jumper wires, so wires were soldered to the module pins and tested one sensor at a time.
- **Servos.** Each servo was mounted on a folded cardboard riser beside the belt, with a layered plastic arm and paddle on its horn. The two servos sit on opposite sides of the belt.
- **Button and buzzer.** Both were soldered with short leads. A stray wire bridging the button's legs once made it read as permanently pressed; removing it fixed the fault.
- **Power and ground.** The wires were joined into the four shared rails of Table 13 and taken to single ESP32 pins.

[Insert photographs of the assembled prototype here.]

5.2 Firmware implementation

The firmware was developed in milestones, each tested on the hardware before the next was added (Table 16).

Table 16: Firmware milestones and their status

Milestone	Content	Status
H1	WiFi connection, IP address, LED heartbeat	Working
H2	Belt forward, backward and stop with PWM speed, controlled from the serial console	Working on hardware
H3	Infrared sensors with debouncing	Working on hardware, both sensors
H4	Servo sweep, rest and push angles	Both servos move; servo 1 verified pushing the box; angles at defaults
H5	Button starts a run; buzzer beeps	Working on hardware
HTTP	Local control server, mDNS name, CORS	Working; verified from a browser

Milestone	Content	Status
H4b	Bench test: button starts the belt, sensor stops it, servo pushes, buzzer beeps	Verified end to end for Bed 1
H6	Full delivery loop with the backend: poll job, button, belt, bed sensor, push, report	Written and compiled; backend side verified by tests; not yet run on the machine

5.3 Backend implementation

The backend is a FastAPI application organised as one router per resource. Request and response shapes are Pydantic models, tables are SQLAlchemy classes, and a dependency checks the caller's token and role before each protected endpoint runs. A WebSocket manager broadcasts delivery, conveyor and alert events to connected browsers. The conveyor endpoints (next-job, dispatch, mark-arrived, status) are called by the firmware with the machine's own technician login. During the project two endpoints were added to close gaps found in the flow: **receive**, with which a patient confirms their own delivery (replacing the nurse's QR scan in the web-only design), and **fail**, with which a technician or the conveyor closes a delivery that cannot complete.

5.4 Web application implementation

The application uses the Next.js App Router with React components. A shared authentication context stores the token and the user, and a protected-page wrapper redirects users to the home page of their own role. Axios attaches the token to every request, and a WebSocket hook keeps dashboards live. The hardware control panel is a React component that calls the conveyor's local routes directly, shows an online or unreachable state, and lets the technician change the conveyor address when the router has assigned a new one.

5.5 Integration

The pieces were connected in this order: the firmware logs in to the backend with the machine's technician account; it asks for the next job; the button press dispatches the job; the belt runs to the bed sensor; the servo pushes; the firmware marks the delivery arrived; and the patient confirms receipt from the web application. The web application and the conveyor were first integrated through the hardware control panel, which drove the real belt, servo and buzzer over WiFi and completed a run for Bed 1.

5.6 Problems encountered and solutions

Table 17: Problems encountered and how they were solved

Problem	Cause	Solution
The belt buzzed and did not turn at low speed	The ENA jumper tied the enable input high, and a very high PWM frequency was used	Removed the jumper, set the PWM frequency to 1 kHz and the speed to 90 of 255
The IR sensors' signal was unreliable	The small connector did not grip jumper wires	Soldered wires directly to the sensor pins
Too few ground, 5 V and 3.3 V pins	Seven parts needed power and ground	Four shared rails with the grounds split by group
The belt sometimes did not move	Motor wires worked loose in the terminals	Re-fitted the wires fully, strain-relieved them, and tested the belt before every trial
The button read as permanently pressed	A wire bridged its two legs	Removed the bridging wire
The board's and laptop's addresses kept changing	The router reassigns addresses	Look the devices up by name (mDNS) with an editable fallback address
Stopped runs stayed 'in transit'	No way to close a delivery that could not finish	Added the fail endpoint, a Mark failed button, and a fail call from the firmware
Web-only design had no confirmation step	Confirmation needed a nurse's QR scan	Added a patient 'I received it' action

Chapter 6: Testing and Results

6.1 Test strategy

Testing was done at three levels: bring-up tests of each hardware part on its own, automated end-to-end tests of the backend, and trials of the assembled machine. Results are reported as they were observed; items not yet tested on the hardware are listed as such.

6.2 Backend tests

Seven automated tests run complete workflows against a test database using pytest. All seven pass.

Table 18: Automated backend tests

Test	What it checks
Full dispensing workflow	Accounts, patient registration, prescription, delivery, conveyor status and alerts across roles
Patient sign-up and own-data access	A patient can register, sees only their own records, and cannot reach staff data
Consultation flow	A patient requests a doctor, the doctor accepts and prescribes, and the consultation completes
Minimal conveyor flow	The conveyor's own sequence: next job, dispatch, a refused second dispatch, and mark arrived
Next job is first in, first out	With two beds waiting, the oldest job is handed out first
Patient confirms receipt	Only the delivery's own patient can confirm, only after arrival, and the prescription becomes delivered
Technician can fail an open delivery	An unfinished delivery can be closed as failed, finished ones cannot, and other roles are refused

6.3 Hardware tests

Table 19: Hardware tests and observations

Test	Result
Belt motor control	Runs steadily and quietly at speed 90 of 255 once the motor wiring is secure; start and stop respond over serial and WiFi
Infrared sensors A and B	Both react cleanly to a hand and to the box; no false events with nothing in front of them
Servos 1 and 2	Both move on command; servo 1, mounted at Bed 1, pushes the box off the belt
Push button	Each press and release is reported once; the button rests open after the wiring fault was fixed
Buzzer	Sounds on command and on button press and delivery

Test	Result
Bed 1 trial: sensor, belt and servo	Five confirmed clean pushes; the sensor stopped the belt 2.4 to 2.9 s after the start in the normal runs, and 5.8 s in one slower run
Button-started run	One press started the belt; the box was detected, pushed off, and the delivered beep sounded
Web-controlled run	Start, Stop, Push Bed 1 and Beep from the web page worked on the real machine, and a web-started run completed for Bed 1
Bed 2: sensor and servo	Sensor B and servo 2 were verified individually and together in a bench test; the belt trial for Bed 2 is pending

6.4 Results against the objectives

Table 20: Objectives and their status

Objective	Status
1. Conveyor carrying a box to one of two beds	Built; belt verified; Bed 1 verified on the belt; Bed 2 trial pending
2. Detect the box and push it into the correct chute	Verified for Bed 1; Bed 2 components verified separately
3. Backend with a documented API	Complete; seven tests pass; interactive documentation generated by FastAPI
4. One web application for four roles	Complete; type-checked and linted; patient, doctor, technician and administrator pages built
5. Conveyor connected to the backend	Firmware written and compiled; backend contract verified by tests; the first full run on the machine is pending
6. Safe behaviour	Emergency stop, timeout and failed-delivery handling implemented; the failure paths of the firmware have not been run on the hardware
7. Verification by tests and trials	Seven automated tests and the hardware trials above

6.5 Limitations and known issues

- The complete delivery loop (H6) has been written and compiled but has not yet been run on the machine with a real prescription.
- Bed 2 has not yet been run on the belt; only its sensor and servo have been tested.
- The servo rest and push angles are still the default values and need calibrating on the mounted arms.
- The conveyor's local control routes use plain HTTP with no password and are meant for a private network.
- The backend's default administrator password must be changed before wider use.

- The system carries one box at a time and serves two beds at model scale; it has not been validated for use with real medicines.

6.6 Discussion

The results show that the chosen design is simple enough to build and test reliably. The parts that failed during development were almost all physical connections, such as loose wires, a bridged button and a poor sensor connector, and each was found by testing one part at a time. Splitting the work into milestones and adding automated tests for the backend made it possible to locate faults quickly. The remaining work is mostly bench work: running the final firmware with a real prescription, running Bed 2 on the belt, and calibrating the servos.

Chapter 7: Conclusion and Future Scope

7.1 Conclusion

MediDispense shows that a low-cost, networked machine can take a prescription from a doctor to the correct bed and record every step. A conveyor with one pusher and one sensor per bed replaces the navigation problems of a mobile robot, an ESP32 connects the machine to a FastAPI backend, and one Next.js web application serves patients, doctors, technicians and administrators. The backend is verified by seven automated tests, and delivery to Bed 1 has been verified on the hardware. The remaining steps are to run the complete delivery loop with a real prescription, to run Bed 2 on the belt and to calibrate the servos.

7.2 Future scope

- A WiFi setup page so the conveyor can join a new network without being reflashed.
- Fixed network addresses for the server and the conveyor.
- Servo angles saved and adjusted from the web application.
- More beds, each with its own pusher and sensor, and a longer belt.
- Optional QR verification of the medicine and the patient, which the backend already supports.
- Push notifications to phones when a delivery arrives.
- HTTPS, authentication for the device itself, and a MySQL deployment for production use.
- A proper enclosure, a power supply that runs the ESP32 from the 12 V adapter, and a nurse interface.

References

[1] FastAPI documentation, https://fastapi.tiangolo.com

[2] Next.js documentation, Vercel, https://nextjs.org/docs

[3] SQLAlchemy documentation, https://docs.sqlalchemy.org

[4] Arduino-ESP32 documentation, Espressif Systems, https://docs.espressif.com/projects/arduino-esp32/

[5] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)", RFC 7519, Internet Engineering Task Force, 2015.

[6] I. Fette and A. Melnikov, "The WebSocket Protocol", RFC 6455, Internet Engineering Task Force, 2011.

[7] S. Cheshire and M. Krochmal, "Multicast DNS", RFC 6762, Internet Engineering Task Force, 2013.

[8] STMicroelectronics, "L298 Dual Full-Bridge Driver", product datasheet.

[9] Pydantic documentation, https://docs.pydantic.dev

[10] pytest documentation, https://docs.pytest.org

Appendix A: Principal API Endpoints

Table 21: Principal API endpoints

Method and path	Called by	Purpose
POST /auth/login	Anyone	Log in and receive a token
POST /auth/signup/patient	New patient	Create a patient account and record in one step
GET /doctors	Patient	List doctors and their availability
POST /consultations	Patient	Request a consultation
POST /consultations/{id}/accept	Doctor	Accept a request
POST /prescriptions	Doctor	Write a prescription; creates a delivery
GET /deliveries	Any signed-in user	List deliveries (patients see only their own)
GET /conveyor/next-job	Conveyor (technician login)	Oldest preparing delivery, or none
POST /deliveries/{id}/dispatch	Conveyor	Button pressed: preparing to in transit
POST /deliveries/{id}/mark-arrived	Conveyor	After the push: in transit to arrived
POST /deliveries/{id}/receive	Patient	Patient confirms receipt: arrived to confirmed
POST /deliveries/{id}/fail	Technician, admin, conveyor	Close an open delivery as failed
POST /conveyor/simulate	Technician, admin, conveyor	Report or simulate the belt status; raises jam alerts
GET /conveyor/status	Any signed-in user	Current belt status
GET /alerts, POST /alerts/{id}/acknowledge, /resolve	Staff	View, acknowledge and resolve alerts
GET /reports/summary	Staff	Summary figures for the reports page
WebSocket /ws/{channel}	Web application	Live events for the conveyor, alerts and prescriptions

Appendix B: Firmware Controls

Table 22: Serial console commands used during development

Key	Action
f, b, s	Run the belt forward, backward, or stop
+, -	Faster or slower
1, 2	Select servo 1 or servo 2
. and ,	Servo angle up or down
z, x	Save the current angle as rest or push
p	Run a push on the selected servo
k	Beep
i	Show both infrared sensor levels
t	Toggle the bench test (sensor pushes its own servo)
h	Toggle the full delivery loop
?	Show the help

Table 23: Firmware constants

Setting	Value
Belt PWM frequency and speed	1 kHz, 90 of 255
Sensor sampling and debounce	every 10 ms; 5 agreeing samples
Job polling interval	4 seconds
Run timeout	40 seconds
Backend call timeouts	3 s connect, 5 s read
Servo rest and push angles (defaults)	0 and 90 degrees
Token lifetime	720 minutes